پروژه فاز دوم-تحقیق در عملیات ۲

دانشکده مهندسی صنایع

متین بقائی
۴۰۲۱۰۳۹۲۹

سلوی دانشور
۴۰۲۱۰۴۰۵۸

بهسا دلشاد
۴۰۲۱۰۴۰۶۹

پردیس عالی داعی
۴۰۲۱۰۴۳۱۴

فهرست مطالب

قسمت ب ۳

بخش اول : استفاده از کتابخانه Pyomo ۳

بخش دوم : استفاده از کتابخانه Gurobipy ۵

قسمت ج ۷

بخش اول: تحلیل حساسیت مقادیر سمت راست با استفاده از کتابخانه pyomo ۷

بخش دوم: تحلیل حساسیت ضرایب تابع هدف با استفاده از کتابخانه pyomo ۹

بخش سوم : تحلیل حساسیت ضرایب تابع هدف با استفاده از کتابخانه Gorubipy.....۱۱

بخش چهارم: تحلیل حساسیت مقادیر سمت راست با استفاده از کتابخانه Gurobipy.....۱۳

قسمت د.....۱۵

بخش اول: رسم نمودار برای تاثیر مقادیر سمت راست بر مقدار تابع هدف با استفاده از کتابخانه pyomo.....۱۵

بخش دوم : رسم نمودار برای تاثیر ضرایب تابع هدف بر مقدار تابع هدف با استفاده از کتابخانه pyomo.....۱۶

بخش سوم : رسم نمودار برای تاثیر تغییر ضرایب تابع هدف با استفاده از کتابخانه Gorubipy.....۱۷

بخش چهارم: رسم نمودار برای تاثیر مقادیر سمت راست بر مقدار تابع هدف با استفاده از کتابخانه Gurobipy ۱۸

قسمت ب

بخش اول: استفاده از کتابخانه Pyomo

(فایل کد مربوط به این بخش به نام pyomo_model در پوشه ای به نام "قسمت ب" ذخیره شده است)

برای نوشتن این کد، لازم است تا پس از ایمپورت کردن کتابخانه pyomo، محدودیت ها به همان شکلی که در بخش اول نوشته شده بود، در کد نیز پیاده سازی شوند. ابتدا یک بازه[۱] برای ساعت ها (از ساعت ۰ تا ۲۳) تعریف شده تا به عنوان اندیس برای متغیر ها به کار روند. در مرحله بعد متغیر های مسئله تعریف شده اند، و سپس محدودیت های مسئله در کد وارد شده اند. اولین محدودیت نوشته شده مربوط به ظرفیت حداکثری ۱۰۰ کارمند برای هر شیفت است ، سپس شیفت هایی که در هر ساعت مشغول به کار هستند در یک لیست تو در تو[۲] ذخیره شده اند. از هر کدام از آیتم های این لیست در محدودیت های مربوط به حداکثر و حداقل کارکنان در هر ساعت (حداکثر ۴۲۰ و حداقل ۸۰ کارمند) و محدودیت های مربوط به تقاضا و کمبود استفاده شده است . سپس برای هرکدام از متغیر های کمبود مطابق خواسته مسئله حد بالای ۱۰ در نظر گرفته شده است ؛ همچنین برای مجموع کمبود های کارمندان نیز حد بالاهای ۴۰ و ۳۰ اعمال شده اند.

برای نوشتن تابع هدف نیز، یک لیست weights تعریف شده که ضرایب تابع هدف را ذخیره می‌کند. سپس خود تابع هدف نوشته شده که از عناصر این لیست استفاده می‌کند و هر عنصر را در متغیر مربوط به آن ضرب می‌کند، همچنین متغیر های مربوط به کمبود و ضرایبشان نیز در تابع هدف وارد می‌شوند.

در نهایت از سالور cbc استفاده می‌کنیم، زیرا این سالور برای مسائل برنامه‌ریزی خطی و عدد صحیح طراحی شده و دقیقاً با ساختار مدل ما سازگار است.

پس از حل مدل، جواب بهینه برابر با ۱۸۴۸۴۸ خواهد بود، و مقادیر متغیر ها مطابق با جدول زیر هستند (البته توجه شود این کد جواب بهینه چندگانه دارد، و این فقط یکی از جواب‌های آن است. امکان دارد در هر بار ران کردن مجدد کد، مقدار برخی متغیرها متفاوت از هم باشد، اما نهایتا مقدار تابع هدف یکسان خواهد بود):

[۱] range
[۲] Nested list

متغیر / اندیس	X_i	Y_i	S^+_i	S^-_i	S'^+_i	S'^-_i
۰	۰	۰	۷	۰	۰	۰
۱	۰	۴	۰	۳	۰	۹
۲	۰	۰	۰	۰	۰	۰
۳	۰	۱۳	۳	۰	۰	۰
۴	۶	۲۴	۰	۰	۰	۰
۵	۳۳	۰	۹	۰	۱	۰
۶	۲۱	۲۹	۰	۰	۱۷	۰
۷	۵۶	۴۴	۰	۰	۰	۰
۸	۵۷	۰	۰	۰	۰	۰
۹	۴۸	۲۹	۰	۰	۰	۰
۱۰	۱۶	۲۸	۱	۰	۰	۰
۱۱	۴۳	۰	۳۷	۰	۰	۰
۱۲	۴۷	۴۹	۰	۰	۰	۰
۱۳	۲	۳۸	۳۳	۰	۰	۰
۱۴	۵۰	۲۳	۰	۰	۴۷	۰
۱۵	۷۱	۲۹	۰	۰	۱۷	۰
۱۶	۲۶	۳۱	۳	۰	۰	۰
۱۷	۶۱	۲۳	۰	۱۰	۰	۱۰
۱۸	۴۰	۳۰	۰	۴	۰	۶
۱۹	۲۶	۲۴	۰	۰	۰	۳
۲۰	۳۷	۳	۰	۱۰	۰	۰
۲۱	۲۹	۰	۰	۰	۰	۰
۲۲	۲۴	۱۳	۰	۳	۰	۰
۲۳	۰	۰	۰	۱۰	۰	۰

بخش دوم: استفاده از کتابخانه Gurobipy

(فایل کد مربوط به این بخش به نام Gurobipy _model در پوشه ای به نام "قسمت ب" ذخیره شده است.)

در این بخش ابتدا کتابخانه‌ی گروبیپای را ایمپورت می‌کنیم و ادامه‌ی کد را مطابق با توضیحات مربوط به نوشتن کد پایامو که در بخش قبل توضیح داده شد می‌نویسیم با این تفاوت که نیازی به استفاده از سالورهای مختلف نیست و از خود کتابخانه‌ی گروبیپای می‌توان استفاده کرد.

در این بخش با اینکه جواب بهینه مطابق با انتظار با بخش قبل یکی می‌شود اما مقدار متغیرها متفاوت می‌شود که ما را به این نتیجه می‌رساند که این مساله دارای جواب بهینه ی چندگانه است.

پس از حل مدل مانند قسمت قبل به جواب بهینه ی ۱۸۴۸۴۸ می‌رسیم البته با این تفاوت که متغیرهایی که مقدار صفر دارند و متغیرهایی که مقدار غیر صفر دارند مطابق جدول صفحه ی بعد تغییر کرده‌اند که این به علت موضوعی است که به آن اشاره شد.

متغیر / اندیس	X_i	Y_i	S^+_i	S^-_i	S'^+_i	S'^-_i
۰	۰	۳	۱۰	۰	۰	۰
۱	۰	۴	۰	۳	۰	۷
۲	۰	۱۱	۰	۰	۰	۰
۳	۰	۱۳	۱	۰	۰	۰
۴	۶	۲۳	۰	۰	۰	۰
۵	۳۱	۰	۷	۰	۴	۰
۶	۲۳	۱۸	۰	۰	۲۰	۰
۷	۵۶	۴۴	۰	۰	۱	۰
۸	۵۵	۱	۱	۰	۰	۰
۹	۵۲	۲۹	۰	۰	۰	۰
۱۰	۱۴	۳۹	۲	۰	۰	۰
۱۱	۴۳	۰	۴۰	۰	۰	۰
۱۲	۴۹	۴۸	۱	۰	۰	۰
۱۳	۰	۳۸	۳۶	۰	۰	۰
۱۴	۵۰	۱۰	۱	۰	۴۵	۰
۱۵	۷۱	۲۹	۰	۰	۱۵	۰
۱۶	۲۴	۳۴	۱	۰	۱	۰
۱۷	۶۳	۲۱	۰	۱۰	۰	۱۰
۱۸	۴۰	۴۳	۰	۴	۰	۶
۱۹	۲۶	۲۴	۱	۰	۰	۵
۲۰	۳۹	۰	۰	۱۰	۱	۰
۲۱	۲۷	۲	۱	۰	۱	۰
۲۲	۲۴	۰	۰	۳	۱	۰
۲۳	۰	۰	۰	۱۰	۱	۰

قسمت ج

بخش اول: تحلیل حساسیت مقادیر سمت راست با استفاده از کتابخانه Pyomo

(کدهای نوشته شده برای این بخش در پوشه تحلیل حساسیت مقادیر سمت راست پایومو ذخیره شده اند.)

برای حل این بخش، ابتدا این ایده به ذهن می‌رسد که مدل یکبار حل شود و مقادیر متغیرها ذخیره شوند، و در مراحل بعدی یک حلقه نوشته شود و در هر مرحله مقادیر متغیرها با هم مقایسه شوند. هنگامی که این مقادیر متفاوت بود حلقه متوقف شده و کران بالا یا پایین گزارش شود. اما در این کد به دلیل داشتن چندین جواب بهینه امکان چنین رویکردی وجود ندارد. زیرا ممکن است solver در مرتبه اول حل به یکی از جواب‌ها برسد، اما در حل مجدد یکی دیگر از پاسخ ها را گزارش کند (در حالی که پاسخ قبلی همچنان صدق می‌کرده) و حلقه را به اشتباه متوقف کند. همچنین ما کنترلی روی این نداریم که solver در مرتبه اول کدام جواب را پیدا می‌کند ، و بنابراین کران‌های به دست آمده در هر بار حل با توجه به هر جواب متفاوت خواهد بود و امکان اعتبارسنجی آن وجود نخواهد داشت .

به همین دلیل از رویکرد دیگری استفاده می‌کنیم. به این ترتیب که ابتدا یک تابع برای حل مدل می‌نویسیم که ورودی آن مقدار سمت راست محدودیت مورد نظر باشد؛ و سایر بخش‌ها کاملا مشابه مدل هستند. سپس تابع is_feasible را تعریف می‌کنیم که خروجی آن یک boolean است. در صورتی که یک جواب در فضای بهینه مسئله صدق کند true، و در غیر این صورت خروجی آن false خواهد بود. در مرحله بعد تابع compute_objective تعریف می‌شود، که مشخصا یک جواب را در تابع هدف قرار می‌دهد و مقدار آن را به عنوان خروجی برمی‌گرداند. پس از تعریف توابع یکی از جواب‌ها را که از پیش تعیین کرده‌ایم (و در قسمت حل مدل با پایومو گزارش شده است) را به مدل می‌دهیم. در هر مرحله از این حلقه مقدار سمت راست به اندازه یک واحد تغییر می‌کند. در اثر این تغییر، ابتدا بررسی می‌شود که آیا جوابی که ابتدا به دست آوردیم، در فضای جواب مسئله صدق می‌کند یا خیر. در صورتی که صدق نکند حلقه متوقف شده و کران بالا یا پایین گزارش می‌شود. در صورتی که صدق بکند، مدل با مقدار جدید محدودیت سمت راست حل شده و مقدار تابع هدف گزارش می‌شود. اگر مقدار تابع هدف بهتر بود، حلقه متوقف می‌شود و کران گزارش می‌شود. در غیر اینصورت حلقه یک مرحله جلوتر می‌رود و مجددا مقدار سمت راست یک واحد تغییر می‌کند.

تعریف حلقه و مسئله به این شکل سبب می‌شود تا تضمین شود مدل همواره روی یک جواب کار می‌کند، تا جایی که واقعا آن جواب دیگر صدق نکند یا که جواب بهینه نباشد. تعریف حلقه به این شکل سبب می‌شود تا رفتار مدل کنترل شود و از پرش بی‌دلیل روی سایر جواب‌های بهینه اجتناب می‌کند. همچنین چون همواره تحلیل حساسیت از یک جواب بهینه ثابت شروع می‌شود ، حتما در هر بار حل یک جواب معین گزارش می‌شود .

مقادیر به دست آمده به شرح زیر هستند (ترتیب فایل‌ها در پوشه به ترتیب زیر است):

- دسته محدودیت های کوچکتر مساوی ۱۰۰ برای تعداد اپراتور های هر شیفت – اندیس ۷ :
  حد پایین = ۱۰۰ حد بالا = ۱۰۱
- دسته محدودیت های کوچکتر مساوی ۴۲۰ برای تعداد اپراتورهای هر ساعت – اندیس ۱۰:
  حد پایین = ۲۹۱ حد بالا = بی کران (چون محدودیت کامل براورده نشده بود)
- دسته محدودیت های بزرگتر مساوی ۸۰ برای تعداد اپراتورهای هر ساعت – اندیس۱۰ :
  حد پایین = ۰ حد بالا = ۲۹۱
- دسته محدودیت های مربوط به تقاضا و لیست demand برای متغیر x – اندیس ۶ :
  حد پایین = ۶۰ حد بالا = ۶۰
- دسته محدودیت های مربوط به تقاضا و لیست demand برای متغیرy – اندیس ۶ :
  حد پایین = ۴۰ حد بالا = ۴۰
- دسته محدودیت های مربوط به حد بالای متغیر کمبود برای متغیر x – محدودیت ۲۰ ام :
  حد پایین =۱۰ حد بالا = ۱۰
- دسته محدودیت‌های مربوط به حد بالای متغیر کمبود برای متغیر y - محدودیت ۱۷ ام :
  حد پایین = ۱۰ حد بالا = ۱۰
- دسته محدودیت های مربوط به حد بالای مجموع کمبود های متغیر x :
  حد پایین = ۴۰ حد بالا = ۴۰
- دسته محدودیت های مربوط به حد بالای مجموع کمبودهای متغیر y :
  حد پایین =۲۸ حد بالا = ۳۰

بخش دوم: تحلیل حساسیت ضرایب تابع هدف با استفاده از کتابخانه Pyomo

(کدهای نوشته شده برای این بخش در پوشه تحلیل حساسیت ضرایب تابع هدف پایومو ذخیره شده اند)

ایده اولیه در اینجا نیز مشابه بخش اول بوده است اما چیزی که باعث شد تحلیل حساسیت را به گونه ای انجام ندهیم که اگر در یک حل به یک پایه بهنیه و در حله دیگر به پایه بهینه دیگر رود این بود که ممکن است هزاران ترکیب مختلف از x و y(جواب‌های چندگانه) وجود داشته باشد، اما تنها یک مقدار بهینه برای تابع هدف وجود دارد.

یعنی اگر کمترین هزینه ممکن ۱۰۰۰ تومان باشد، محال است حل‌گر جوابی با هزینه ۱۰۰۰ پیدا کند و یک ثانیه بعد جوابی با هزینه ۹۹۹ پیدا کند (مگر اینکه پارامترها عوض شوند).

حالتی را در نظر میگیریم که ضریب تابع هدف ۱۸۹ و برای متغیر های x باشد و حالت فرضی برای ساده سازی:

هزینه هر نفر:۱۸۹ $x_{۲۰} = ۵$ **کل هزینه:** ۱۰۰۰۰ واحد

حال درحلقه تحلیل حساسیت، هزینه آن ۵ نفر را می‌کنیم **۱۹۰(یک واحد افزایش)**

انتظار ریاضی ما: ۱۰۰۰۰ + ۵*۱=۱۰۰۰۵

حالا حل‌گر را اجرا می‌کنید. دو حالت پیش می‌آید:

حالت ۱: پرش به جواب هم‌ارز

حل‌گر به جای ترکیب قبلی، یک ترکیب جدید پیدا می‌کند (مثلاً شیفت‌های دیگر را تغییر می‌دهد) اما همچنان هزینه نهایی‌اش همان **۱۰۰۰۵** می‌شود.

نتیجه کد: شرط obj < expectedچک می‌شود (آیابرقرار است؟ خیر) پس حلقه ادامه می‌یابد. یعنی درست کار کرد. چون هزینه دقیقاً طبق انتظار بالا رفته، پس هنوز در بازه ثبات هستیم.

حالت ۲: پیدا شدن راه واقعاً ارزان‌تر (خروج واقعی)

حل‌گر متوجه می‌شود که با قیمت ۱۹۰ تومان، دیگر نمی‌صرفد ۵ نفر استخدام کند. پس ساختار را عوض می‌کند و مثلاً ۴ نفر استخدام می‌کند و کمبود را جور دیگری جبران می‌کند.

- هزینه واقعی حل‌گر: ۱۰۰۰۲ (مثلاً)
- انتظار ما: ۱۰۰۰۵
- نتیجه کد: شرط ۱۰۰۰۵ > ۱۰۰۰۲ صحیح می‌شود. پس حلقه می‌شکند. یعنی واقعاً استراتژی عوض شده.

در ادامه تحلیل حساسیتی که انجام شده است آورده میشود. همانطور که در کد مشاهده میکنید ۴ مدل ضریب در تابع هدف داریم یکی ضرایب برای هزینه های دستمزر کارمندان،هزینه های کمبود شرکت سرمایه پارسیان از ساعت ۷ صبح تا ۶ بعد از ظهر(۶۰ تومان)، هزینه های کمبود شرکت سرمایه پارسیان از ساعت ۶ بعد از ظهر تا ۷ صبح (۳۰ تومان) که در کد جریمه شب به آن ذکر شده است و هزینه های کمبود شرکت مهر ایرانیان(۴۵ تومان) را شامل می‌شود.

برای تحلیل حساسیت میتوان هر کدام از ضرایب را بدلخواه در جایی که ذکر شده در خط های حدودا ۱۰۰ index_to_change =٤ تحلیل حساسیت ضریب آن متغیر را انجام داد. دقت شود که برای تحلیل حساسیت ضرایب دستمزد کارمندان base_weight = ۱۷۱ نیز باید تغییر گردد.

فرآیند هایی که در کد اجرا می‌شود:

۱. پارامتری‌سازی مدل(Mutable Parameters)

در ابتدا، ضرایب تابع هدف (دستمزدها یا جریمه‌ها) به جای عدد ثابت، به صورت Paramبا ویژگی mutable=True تعریف شده‌اند.

این کار به ما اجازه می‌دهد در تابع solve_with_weight بدون بازسازی کل مدل، مقدار ضریب را با دستور set_value()تغییر دهیم.

۲. حلقه جستجوی تحلیل حساسیت

کد از دو حلقه whileبرای حرکت روی محور اعداد (بالا و پایین ضریب فعلی) استفاده می‌کند.

منطق توقف: در هر تکرار، مدل دوباره حل می‌شود. کد به جای مقایسه تک‌تک متغیرها، از شرط انحراف هزینه استفاده می‌کند. اگر تابع هدف حل‌گر (obj) کمتر از مقدار مورد انتظار ما (expected_obj) باشد، یعنی مدل یک راه بهینه جدید پیدا کرده و مرز حساسیت شناسایی شده است.

۳. تحلیل تفاوت جواب‌ها

پس از خروج از حلقه‌ها، تابع compare_solutionsفراخوانی می‌شود .این تابع با بررسی تفاضل مقادیر متغیرها در حالت پایه و حالت جدید (با در نظر گرفتن toleranceبرای حذف خطاهای اعشاری)، دقیقاً گزارش می‌دهد که خروج از بازه حساسیت باعث تغییر در کدام شیفت‌های کاری یا متغیرهای جریمه شده است.

نکته مهم: ما بخطار اینکه ممکن است در سالور سیستم هر شخصی به یک پایه بهینه برسد و خب تحلیل حساسیت برای هر پایه بهینه متفاوت از دیگری است تصمیم گرفتیم که یک پایه بهینه دستی که در سیستم خودم ران شده را برای تحلیل حساسیت در نظر بگیریم تا به مشکل پایه بهینه های چند گانه برنخوریم.

یک مورد انجام شده و در ذیل ذکر شده است:

تحلیل حساسیت ضریب $x_{۴}$: حدپایین:۱۶۵ حدبالا:۱۷۱

تحلیل حساسیت ضریب$s_minus_{۱۷}$ حدپایین:۱۰ حدبالا:۱۱۰

تحلیل حساسیت ضریب جریمه شب $s_minus_{۱}$ حد پایین:۳۰ حدبالا:۵۱

تحلیل حساسیت ضریب $s_prime_minus_{۱۹}$ حدپایین:۴۱ حد بالا:۴۵

بخش سوم : تحلیل حساسیت ضرایب تابع هدف با استفاده از کتابخانه Gorubipy

(کدهای نوشته شده برای این بخش در پوشه تحلیل حساسیت ضرایب تابع هدف گروبیپای ذخیره شده اند)

۱. رویکرد تحلیل حساسیت

چون مدل چندجوابی بود و تعداد جواب های چندگانه زیاد بودند و همچنین ماهیت مسائل عدد صحیح باعث رفتار پله‌ای در جواب‌ها می‌شود، تحلیل حساسیت کلاسیک برنامه‌ریزی خطی قابل اتکا نبود.

برای تحلیل حساسیت، رویکرد {پایداری ساختار جواب} را اجرا کردم. ایده این بود که به جای اینکه فقط تغییر مقدار تابع هدف را بررسی کنم، ببینم آیا با تغییر ضریب، الگوی تصمیم عوض می‌شود یا نه.

برای اینکار دو مدل را در هر تغییر ضریب حل کردم. یکی مدل آزاد یا اولیه بود که اجازه تغییر ساختار داشت. دیگری مدل محدود شده بود که ساختار جواب مرجع را تحمیل می‌کند.

حالا مدل محدود شده را چطور درست کردم؟ اول مدل اولیه را حل کردم. می‌خواستم ترکیب متغیرهای پایه در تحلیل حساسیت ثابت بماند. در این مسیر، ابتدا ساختار جواب مرجع را از روی خروجی حل استخراج و به‌صورت دستی دسته‌بندی کردم. برای متغیرهای ایکس، شیفت‌های صفر شامل ۰، ۱، ۲، ۳، ۱۳ و ۲۳ بودند و بقیه غیرصفر. برای متغیرهای وای نیز شیفت‌های صفر شامل ۲، ۵، ۱۱، ۲۰ و ۲۳ بودند و باقی شیفت‌ها غیرصفر. سپس در کدنویسی، این ساختار را به‌صورت کران‌های بالا و پایین اعمال کردم. متغیرهایی که در جواب مرجع صفر بودند مجبور می‌شدند صفر بمانند و متغیرهایی که در جواب مرجع غیرصفر بودند مجبور می‌شدند مثبت باقی بمانند. در واقع میخواستم معیار شکست، تغییر در ساختار باشد.

۲. تعریف نقطه شکست

پس از آماده‌سازی این دو مدل، معیار تشخیص «نقطه شکست» را به این صورت تعریف کردم: هر زمان مقدار تابع هدف مدل آزاد به طور معناداری کمتر از مدل محدود شده شود، یعنی مدل آزاد توانسته با تغییر ساختار، راه‌حل ارزان‌تری پیدا کند و در نتیجه ساختار جواب مرجع دیگر پایدار نیست.

برای کاهش احتمال پرش‌های ناخواسته و افزایش دقت، گام تغییر ضرایب را کوچک انتخاب کردم (مثلاً ۰٫۰۱ برای ضرایب شیفت) تا مرز شکست دقیق‌تر شناسایی شود.

۳. تحلیل حساسیت روی هزینه شیفت ها ((C(j)

در بخش اول تحلیل نتایج، تمرکز من روی هزینه شیفت‌ها بود و به طور مشخص شیفت شماره ۶ را بررسی کردم. ضریب پایه این شیفت ۱۵۳ واحد بود. خروجی الگوریتم نشان داد که این شیفت از نظر هزینه «بسیار حساس و ناپایدار» است. در جهت افزایش، حتی با افزایش بسیار جزئی از ۱۵۳ به ۱۵۳.۰۱، مدل آزاد هزینه‌ای کمتر از مدل محدود شده به دست آورد. به زبان ساده، کوچک‌ترین افزایش قیمت کافی بود تا مدل تصمیم بگیرد ساختار

شیفت‌بندی را تغییر دهد و به یک آرایش جدید برسد.. در جهت کاهش، نقطه شکست در حدود ۱۵۱.۲ مشاهده شد و بنابراین بازه پایداری به‌صورت تقریبی ۱۵۱.۲ تا ۱۵۳ به دست آمد.

این بازه بسیار باریک، نشان دهنده این است که برنامه فعلی وابستگی شدیدی به قیمت‌گذاری این شیفت دارد و هر افزایش کوچک می‌تواند ساختار بهینه را بر هم بزند؛ یعنی اگر در دنیای واقعی هزینه این شیفت افزایش یابد، باید انتظار تغییر قابل توجه در برنامه شیفت‌بندی را داشت.

۴. تحلیل حساسیت روی ضرایب جریمه

در بخش دوم، تحلیل حساسیت را روی ضرایب جریمه انجام دادم. تمرکز من روی جریمه ۴۵ واحدی مربوط به کمبود سرویس برای بخش «مهر ایرانیان» بود. برخلاف هزینه شیفت، نتایج نشان داد که ساختار جواب نسبت به تغییرات این جریمه مقاومت خوبی دارد. بازه پایداری برای این پارامتر بین ۳۶ تا ۷۰ واحد به دست آمد. تفسیر مدیریتی این خروجی روشن است: اگر مدیریت تصمیم بگیرد اهمیت کمبود نیرو برای مهر ایرانیان را افزایش دهد و جریمه را تا ۷۰ بالا ببرد، یا در جهت کاهش اولویت آن را تا ۳۶ پایین بیاورد، همچنان همان ترکیب کلی نیروها بهینه باقی می‌ماند و برنامه شیفت‌بندی دچار آشفتگی ساختاری نمی‌شود.

بخش چهارم: تحلیل حساسیت مقادیر سمت راست با استفاده از کتابخانه Gurobipy

(کدهای نوشته شده برای این بخش در پوشه تحلیل حساسیت مقادیر سمت راست گروبیپای ذخیره شده‌اند.)

همان طور که در بخش مدلسازی مشاهده کردیم با ران کردن مدل با استفاده از این کتابخانه به یکی دیگر از جواب های بهینه‌ی مدل می‌رسیم و برای اینکه بتوانیم نتایج قابل مقایسه ای بدست آوریم تصمیم گرفتیم از همان جواب اولیه‌ی بخش کتابخانه‌ی پایامو استفاده کنیم و با استفاده از کدی که در بخش قبل توضیح دادیم برای جلوگیری از پرش به جواب های چندگانه‌ی دیگر و اطمینان از کارکرد تحلیل حساسیت مطابق با توضیحات کتابخانه‌ی پایامو کد را با استفاده از کتابخانه‌ی گروبیپای می‌نویسیم. برای تحلیل حساسیت شماره محدودیت‌های مختلفی را انتخاب کردیم تا تنوع افزایش یابد. همچنین برای اینکه پایه بهینه ثابت بماند علاوه بر توضیحاتی که در بخش اول داده شد دادن جواب اولیه‌ی دستی تضمین می‌کند که پایه‌ی بهینه ثابت بماند و تحلیل حساسیتی درستی را بتوانیم انجام دهیم.

مقادیر کران بالا و پایین برای دسته محدودیت‌های مختلف به شرح زیر است:

- دسته محدودیت های کوچکتر مساوی ۱۰۰ برای تعداد اپراتور های هر شیفت – اندیس ۹:

  حد پایین = ۷۷ حد بالا = بی‌نهایت (زیرا محدودیت کامل برآورده نشده بود)

- دسته محدودیت های کوچکتر مساوی ۴۲۰ برای تعداد اپراتورهای هر ساعت – اندیس ۲۰:
  حد پایین = ۳۹۰ حد بالا = بی کران (چون محدودیت کامل براورده نشده بود)
- دسته محدودیت های بزرگتر مساوی ۸۰ برای تعداد اپراتورهای هر ساعت – اندیس۲۰:
  حد پایین =۰ حد بالا = ۳۹۰
- دسته محدودیت های مربوط به تقاضا و لیست demand برای متغیر x – اندیس ۳:
  حد پایین =۵۰ حد بالا = ۵۰
- دسته محدودیت های مربوط به تقاضا و لیست demand برای متغیر y – اندیس۱۰:
  حد پایین =۱۱۰ حد بالا = ۱۱۰
- دسته محدودیت های مربوط به حد بالای متغیر کمبود برای متغیر x – محدودیت ۱۵ ام:
  حد پایین =۰ حد بالا = بی‌نهایت (چون محدودیت کامل برآورده نمی‌شود.)
- دسته محدودیت های مربوط به حد بالای مجموع کمبود های متغیر x:
  حد پایین = ۴۰ حد بالا = ۴۰
- دسته محدودیت های مربوط به حد بالای مجموع کمبودهای متغیر y:
  حد پایین =۲۸ حد بالا = ۳۰
- دسته محدودیت های مربوط به حد بالای متغیر کمبود برای متغیر y – محدودیت ۱۸ام:
- حد پایین =۶ حد بالا = بی‌نهایت (زیرا محدودیت کامل برآورده نشده است)

علت اینکه از دسته محدودیت های محتلف فقط اندیس یکی را بررسی کردیم این بود که باقی آن‌ها نیز به همین ترتیب تحلیل حساسیت می‌شوند و کد نوشته شده برای بخش تحلیل حساسیت مقدار سمت راست قابلیت تحلیل حساسیت تمام اندیس ها را دارد زیرا که برای هر دسته محدودیت کدی جداگانه نوشته شده است که می‌توان شماره اندیسی را که می‌خواهیم برای هر دسته محدودیت تحلیل کنیم مشخص کنیم و ادامه‌ی کد را اگر لازم بود مطابق اندیس اندکی تغییر دهیم.

قسمت د

بخش اول: رسم نمودار برای تاثیر مقادیر سمت راست بر مقدار تابع هدف با استفاده از کتابخانه Pyomo

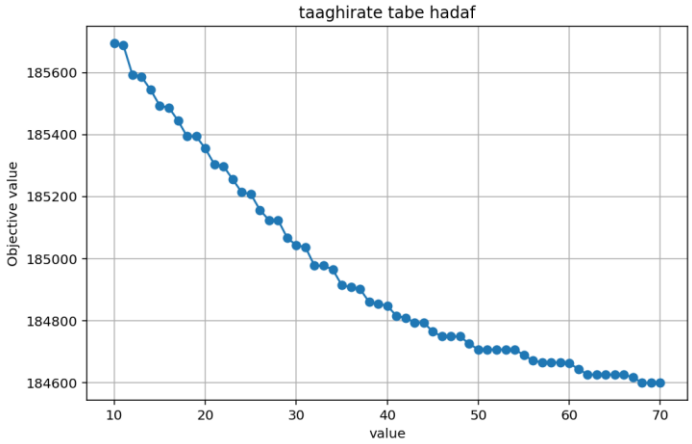
(کد این قسمت در پوشه ای به نام "قسمت د" ، فایل Pyomo_Sensitivity_RHS قرار دارد.)

برای رسم نمودار در این بخش، محدودیت حداکثر میزان کمبود اپراتور موسسه سرمایه پارسیان (که برابر ۴۰ است) انتخاب شده، زیرا با بررسی در قسمت قبل مشخص شد که با تغییر مقدار سمت راست، مقدار تابع هدف نیز تغییر می‌کند و برای نشان دادن تغییرات روی نمودار مناسب است.

در این کد نیز مجددا یک تابع نوشته می‌شود که ورودی آن مقدار متغیر سمت راست محدودیت است. در این تابع مدل را مشابه قسمت ج تعریف می‌کنیم، و خروجی آن نیز مقدار تابع هدف جواب بهینه خواهد بود.

سپس برای مقدار سمت راست متغیر یک بازه تعریف می‌کنیم، و برای مقادیر از ۱۰ تا ۷۰ در یک حلقه کد را اجرا می‌کنیم و هر دفعه مقدار تابع هدف را در لیست objectives ذخیره می‌کنیم. نهایتا با استفاده از کتابخانه matplotlib مقادیر را روی نمودار نمایش می‌دهیم.

انتظار می‌رود هر چه که مقدار متغیر سمت راست کوچکتر شود، فضای بهینه محدود تر شود و در نتیجه به جواب های بدتری (یعنی جواب‌های بزرگتر) برسیم، و هر چقدر که متغیر سمت راست بزرگتر شود جواب‌های بهتری را مشاهده کنیم. نمودار به دست آمده در شکل د-۱ قابل مشاهده است، که دقیقا همین نتیجه‌گیری را نشان می‌دهد.



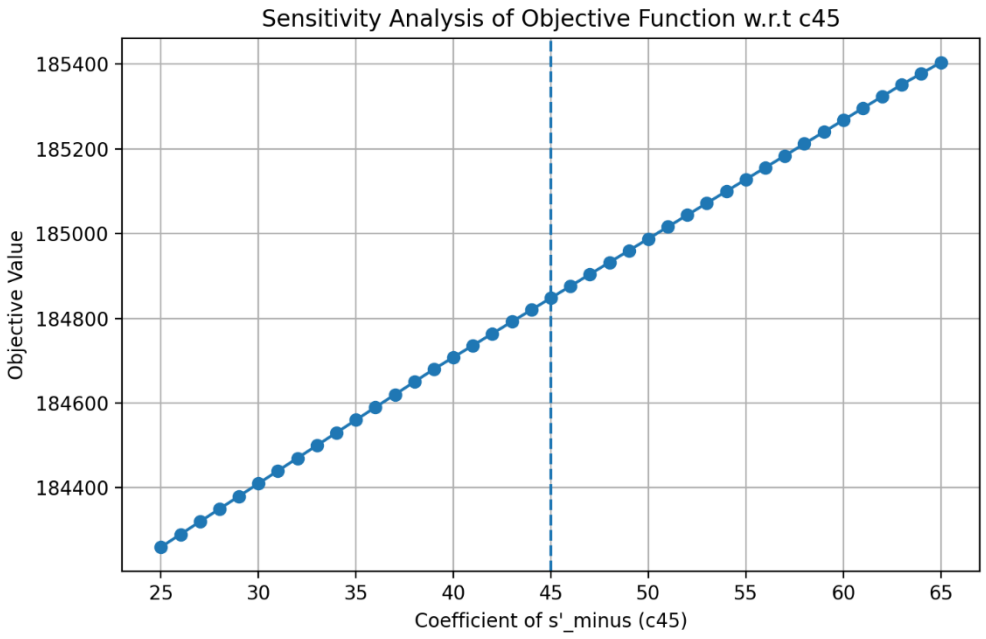
شکل د- ۱ تغییرات مقدار تابع هدف بر اساس تغییر سمت راست محدودیت کمبود اپراتور موسسه سرمایه پارسیان

همچنین می‌توانیم ببینیم که هر چه مقدار متغیر سمت راست بیشتر می‌شود، شیب نمودار کمتر می‌شود و اختلاف تغییرات با افزایش مقدار سمت راست محدودیت کاهش می‌یابد.

بخش دوم : رسم نمودار برای تاثیر ضرایب تابع هدف بر مقدار تابع هدف با استفاده از کتابخانه Pyomo

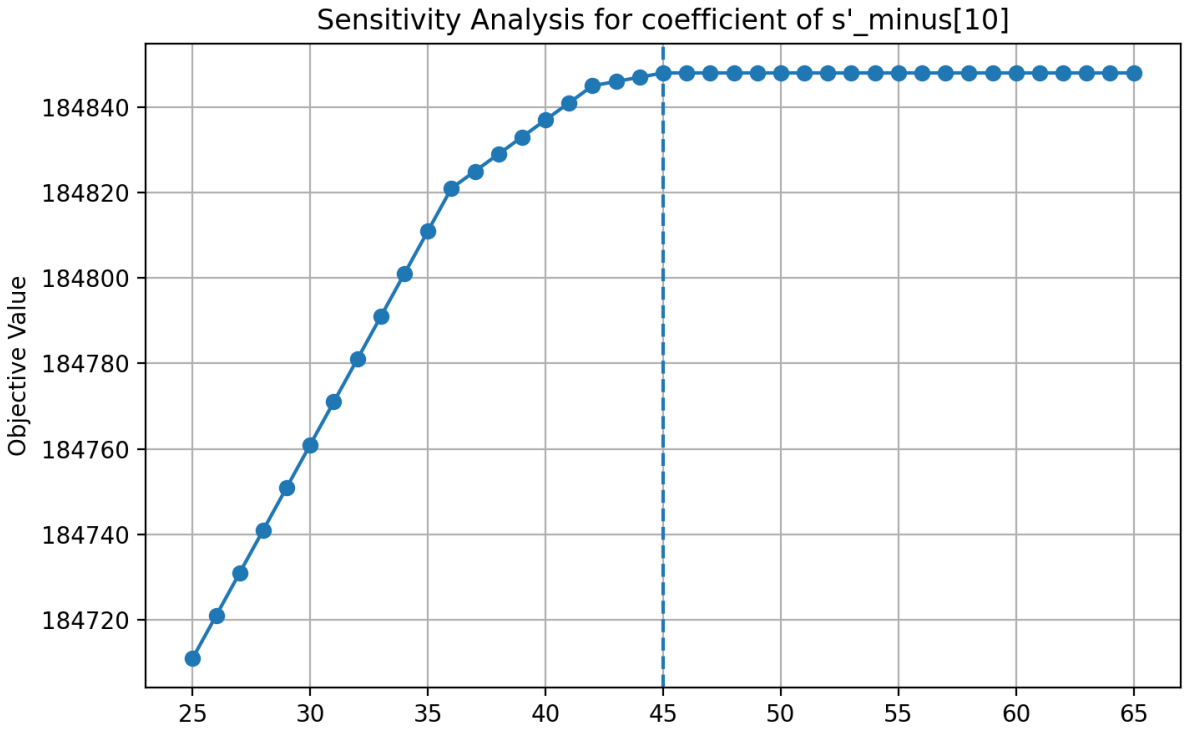
(کد این قسمت در پوشه ای به نام "قسمت د" ، فایل Pyomo_Sensitivity_Coefficient قرار دارد.)

برای رسم این نمودار از ضریب هزینه کمبود شرکت مهر ایرانیان استفاده می‌کنیم که برابر ۴۵ می‌باشد. همانند بخش قبل که برای مقادیر سمت راست بود در این کد تمامی موارد تکرار می‌شود. دقت شود که با توجه به اینکه با افزایش یک ضریب از تابع هدف مقدار تابع هدف افزایش می‌یابد و کاهش ضریب نیز مقدار تابع هدف را کاهش می‌دهد این را هم در نمودار مربوطه مشاهده می‌کنیم. این تحلیل برای ضریب ۴۵ برای هر ۲۴ متغیر کمبودی که شرکت ایرانیان در نظر گرفته است تحلیل شده است.



شکل د-۲.۱ تغییرات مقدار تابع هدف بر اساس تغییر ضریب ۴۵ جریمه کمبود کارمند برای شرکت مهر ایرانیان

و اما تحلیلی که برای یک ضریب از یک متغیر کمبود شرکت ایرانیان بفرض $s_prime_minus_{۱۰}$ که یعنی میزان کمبود در ساعت ۱۰ است و ضریب این متغیر تغییرات آن مقدار تابع هدف را بدنبال دارد. نشان می‌دهد که با کاهش ان مقدار تابع هدف کاهش ولی با افزایش هیچ تغییری در مقدار تابع هدف رخ نمی‌دهد.

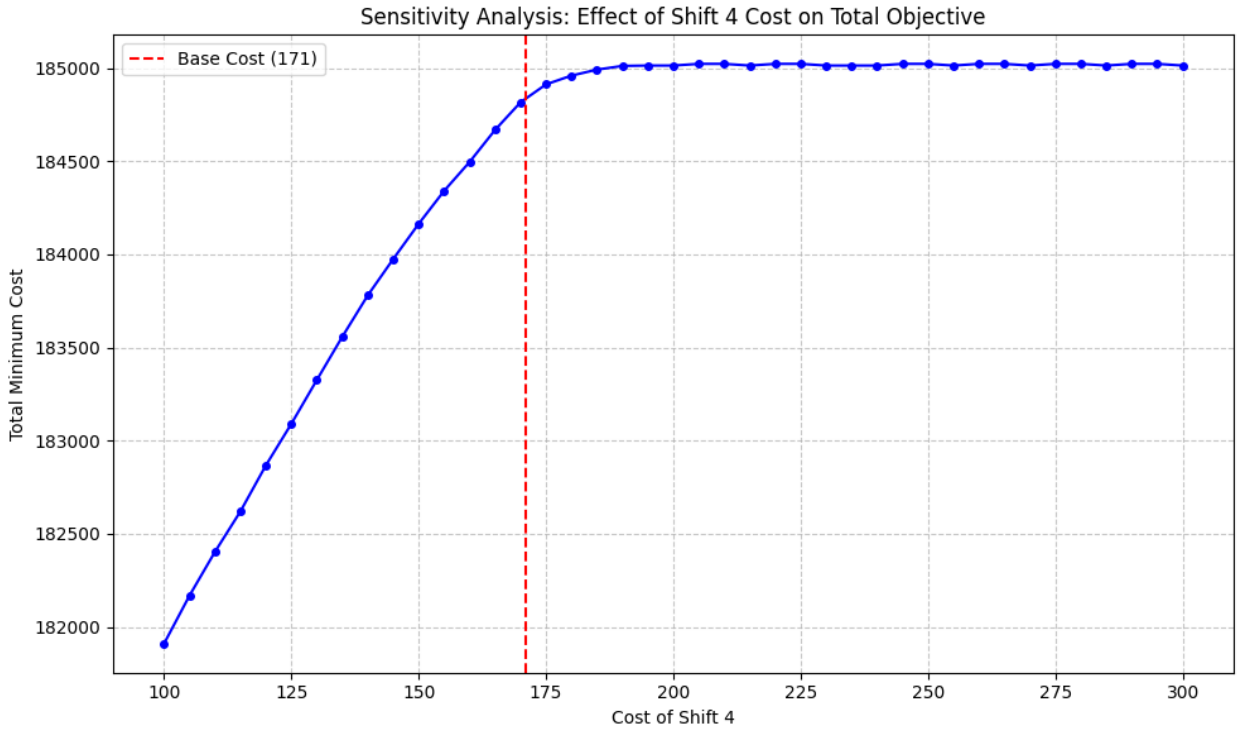


شکل د-۲.۲ تغییرات مقدار تابع هدف بر اساس تغییر ضریب ۴۵ برای یک جریمه کمبود کارمند برای شرکت مهر ایرانیان در ساعت ۱۰

بخش سوم : رسم نمودار برای تاثیر تغییر ضرایب تابع هدف با استفاده از کتابخانه Gorubipy

(کد این قسمت در پوشه ای به نام "قسمت د" ، فایل Gurobipy_Objective_Function قرار دارد.)

هزینه شیفت ۴ را در بازه **۱۰۰ تا ۳۰۰** تغییر دادم و مقدار بهینه تابع هدف را رسم کردم. نمودار نشان داد در قیمت‌های پایین شیفت ۴ اثرگذاری زیادی روی هزینه کل دارد، اما بعد از یک حد مشخص نمودار مسطح‌تر می‌شود؛ یعنی وقتی شیفت خیلی گران می‌شود، مدل استفاده از آن را کم می‌کند و از شیفت‌های دیگر پوشش می‌دهد، پس افزایش بیشتر قیمت اثر کمی روی هزینه کل می‌گذارد.



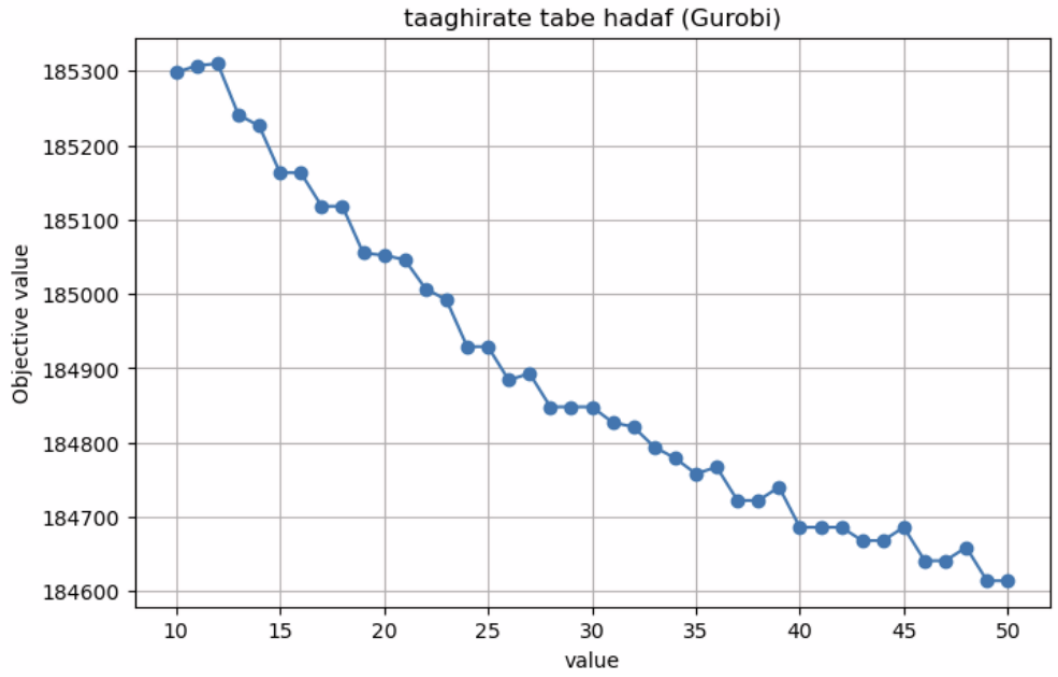
شکل د- ۳تغییرات مقدار تابع هدف بر اساس تغییر ضریب جریمه هزینه شیفت ۴

بخش چهارم: رسم نمودار برای تاثیر مقادیر سمت راست بر مقدار تابع هدف با استفاده از کتابخانه Gurobipy

(کد این بخش در پوشه ای به نام "قسمت د" ، فایل Gurobipy_Sensitivity _RHS قرار دارد .)

مطابق با توضیحات بخش اول یعنی پایامو این بار کد مربوطه را با استفاده از کتابخانه‌ی گروبیپای می‌نویسیم. در اینجا وجود جواب چندگانه برای ما مشکل ایجاد نمی‌کند زیرا که مقدار تابع هدف در اینجا اهمیت دارد و می‌دانیم که مقدار تابع هدف برای تمام جواب های چندگانه یکسان است.

در این بخش مقدار سمت راست مربوط به دسته محدودیت های مربوط به حد بالای مجموع کمبودهای متغیرy (شرکت مهر ایرانیان) را در بازه ی ۱۰ تا ۵۰ رسم کردیم و می‌توان در شکل زیر(د-۴) تغییرات آن را مشاهده کرد. برای اینکار مقدار سمت راست آن را نوشته و یک بازه برای تغییرات آن در نظر می‌گیریم در هر بار مقدار سمت راست در بازه ی مشخص شده افزایش می‌یابد و مدل با مقدار جدید حل می‌شود و جواب را اگر موجه بود در لیست objectives ذخیره می‌کند و اگر غیر موجه بود مقدار none را ذخیره می‌کند و در نهایت در یک نمودار با استفاده از کتابخانه‌ی matplotlib رسم می‌کند. همانطور که در بخش تحلیل حساسیت مشاهده کردیم این محدودیت اصلا در بازه‌ای ثابت نمی‌ماند که بهینه باشد. و همانطور که در نمودار مشاهده می‌شود تقریبا می‌توان گفت با افزایش مقدار سمت راست مقدار تابع هدف کاهش می‌یابد که ما را به جواب‌های بهتری می‌رساند.



شکل د-۴) مربوط به تغییرات تابع هدف مربوط به مقدار سمت راست محدودیت مجموع کمبودهای متغیر y